

Instrucciones

Para cada uno de los siguientes problemas:

- [10%] Seleccione y marque **todas** las propiedades que dicho problema posee.
- [35%] Demuestre que el problema posee dichas propiedades siguiendo los procedimientos vistos en clase (no valen ejemplos como respuestas).
- [30%] Plantee un algoritmo en pseudocódigo para resolver el problema, siguiendo el paradigma correspondiente a las propiedades identificadas en los incisos anteriores.
- [15%] Programe su algoritmo y grabe un video de su algoritmo ejecutando al menos tres (3) casos de prueba no triviales, incluyendo una explicación narrada por usted (no narradores automáticos) acerca de cómo el algoritmo está operando para producir sus resultados. Cargue su video a YouTube como **no listado** y comparta el enlace para su entrega.

Para demostrar las propiedades, recuerde:

- Que la subestructura óptima depende de las **decisiones posibles**, los **subproblemas** que éstas generan; y la **necesaria optimalidad**.
- Que los algoritmos de programación dinámica requieren **caracterizar la solución óptima**, **plantear la relación de recurrencia**; y proveer la **solución apoyada en dicha relación de recurrencia**.
- Que las matroides ponderadas son **parejas de conjuntos** (S, I) donde S es el conjunto de **objetos del problema** (esencialmente, los que conformarán la solución); e I es la **familia de subconjuntos de S** llamados **independientes**, que presentan las propiedades de **herencia** e **intercambio**. La comparación entre los elementos de S se da mediante una **función de peso w** .
- Que la *greedy-choice property* de por sí es un **criterio de decisión** que elige **óptimos locales**, y que su presencia en un **problema de optimización que tiene subestructura óptima** garantiza que el problema **se puede resolver con un acercamiento greedy**.

Debe realizar sus entregas en Canvas, incluyendo:

- Documento PDF con sus respuestas.
- Archivos de código fuente (**además del repositorio**).
- Enlace a repositorio privado de GitHub o BitBucket con el código para sus respuestas. El objetivo del repositorio es evidenciar el progreso de desarrollo.

Problemas

1. **[30 puntos]** Considere el problema de “hacer sencillo”: dado un monto m , buscamos alcanzar dicho monto con la menor cantidad de monedas posible. Las monedas tienen las siguientes denominaciones: $\{1,5,10,25\}$. Podemos tomar todas las monedas que necesitemos de cada denominación.

- ☐ Subestructura óptima
- ☐ Subproblemas traslapados
- ☐ *Greedy-choice property*
- ☐ Se puede modelar como una matroide ponderada

2. **[30 puntos]** *Knapsack problem* fraccionado: el problema visto en clase de un ladrón con una bolsa que aguanta W unidades u de peso, y busca robar el máximo valor que pueda de una selección de n artículos, donde el artículo i tiene precio p_i y hay $w_i u$'s disponibles del mismo. No es necesario robar el artículo completo; es posible tomar porciones del mismo (a nivel de u 's).

- ☐ Subestructura óptima
- ☐ Subproblemas traslapados
- ☐ *Greedy-choice property*
- ☐ Se puede modelar como una matroide ponderada

3. **[40 puntos]** Dado un número entero positivo n y el teclado de un Nokia 3230, que tiene dígitos del 0 al 9 asociados con cada tecla; se desea contar la cantidad total de combinaciones de n dígitos que existen, bajo las siguientes restricciones:
- a. Podemos comenzar con cualquier dígito.
 - b. El siguiente teclazo debe ser arriba, abajo, a la izquierda o a la derecha del teclazo más reciente.
 - c. El teclado también tiene las teclas * y # que no es permitido presionar.

- ☐ Subestructura óptima
- ☐ Subproblemas traslapados
- ☐ *Greedy-choice property*
- ☐ Se puede modelar como una matroide ponderada

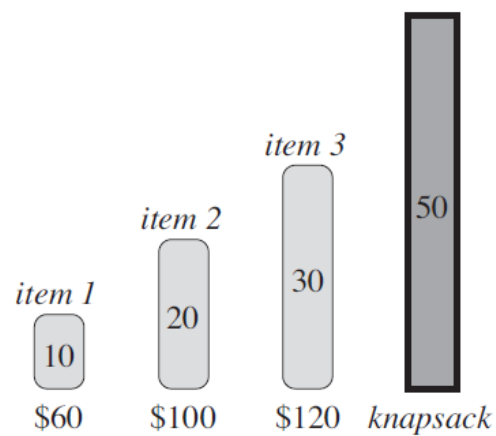
Apéndice

Nokia 3230



Instancias de ejemplo para cada problema:

1. Para hacer sencillo un monto de Q2.93 necesitaríamos: 11 monedas de Q0.25, 1 moneda de Q0.10, 1 moneda de Q0.05, 3 monedas de Q0.01.
- 2.



La solución es 10 unidades del ítem 1, 20 unidades del ítem 2 y 20 unidades del ítem 3.

3.

Teclado:

[' 1 ' , ' 2 ' , ' 3 '],

[' 4 ' , ' 5 ' , ' 6 '],

[' 7 ' , ' 8 ' , ' 9 '],

[' * ' , ' 0 ' , ' # ']

Para $n = 2$, el total de combinaciones posibles son 36:

[00, 08, 11, 12, 14, 21, 22, 23, 25, 32, 33, 36, 41, 44, 45, 47, . . . , 96, 98, 99]